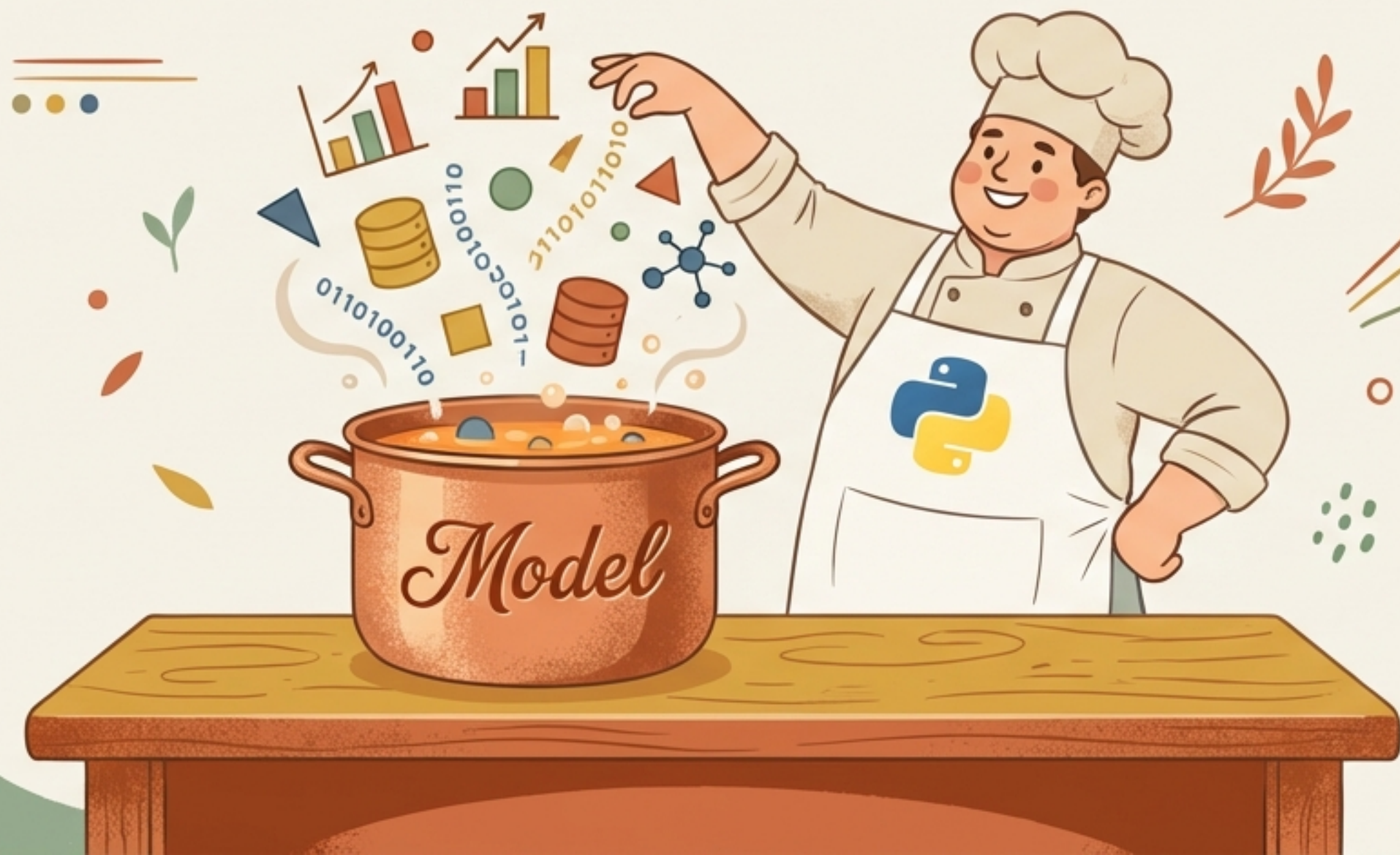


คู่มือเชฟข้อมูลฉบับ Python

จากวัตถุดิบดิบ สู่จานเด็ดแห่ง Insight

ในโลกที่ข้อมูลเพิ่มขึ้นมหาศาล การ ‘ปรุง’ ข้อมูลให้กลายเป็นความรู้ (Insight) คือทักษะที่สำคัญที่สุด เปรียบเสมือนเชฟที่ต้องเปลี่ยนวัตถุดิบดิบให้เป็นอาหารจานหรู



❖ The Menu ❖

❖ Appetizer:

รู้จักอุปกรณ์
(Python Ecosystem)



❖ Prep Work:

เตรียมวัตถุดิบ
(NumPy & Pandas)



❖ Plating:

จัดจานให้สวยงาม
(Visualization)



❖ Main Course:

ปรุงรสและผ่านความร้อน
(Machine Learning)



เตรียม Google Colab ของคุณให้พร้อม
แล้วเริ่มกันเลย!

ทำไมต้อง Python? ขอสลับที่เชฟทั่วโลกเลือกใช้

ขอสลับที่เชฟแห่งการข้อมูล

Python คือภาษาที่มีความอเนกประสงค์ (Versatile) สูง เปรียบเสมือนมีดพับ Swiss Army Knife แห่งวงการข้อมูล ใช้งานได้ตั้งแต่:



งานวิจัย (Research)



สร้างต้นแบบ (Prototyping)



ระบบใช้งานจริง (Production)



****Anaconda: กล่องเครื่องมือสำเร็จรูป****

ไม่ต้องปวดหัวกับการติดตั้งทีละตัว รวม Library สำคัญไว้ให้ครบ เป็นมาตรฐานที่วงการ Data Science ยอมรับ

พื้นที่ทดลองสูตร: Jupyter Notebook

Jupyter Notebook เปรียบเสมือน
"สมุดทดลองดิจิทัล" (Digital Lab Notebook)
ที่ให้คุณทำ 3 อย่างในหน้าเดียว
เพื่อการลองผิดลองถูก (Trial & Error):



Write Code



Run



View Result

```
# พิมพ์คำสั่งนี้ใน Colab เพื่อทักทายโลก  
print("Hello, Data Chef!")
```

Hello, Data Chef!

พื้นที่เขียนสูตร
(Code)

ผลลัพธ์ทันที
(Output)

NumPy: รากฐานแห่งการคำนวณ

THE TOOL



Concept: Library ที่เป็นแกนหลักของการคำนวณทางวิทยาศาสตร์

Analogy: ถ้า List คือการหั่นผักทีละชิ้น NumPy คือร่อนขั้วเห็บทางวิทยาศาสตร์

Analogy: ถ้า List คือการหั่นผักทีละชิ้น NumPy คือเครื่องหั่นอัตโนมัติที่จัดการวัตถุดิบทั้งกองได้ในพริบตา



เร็วกว่า List ปกติของ Python หลายเท่า

THE CODE

Micro-Challenge: สร้าง Array จำลองวัตถุดิบ

```
import numpy as np

# สร้าง Array (วัตถุดิบ)
data = np.array([1, 2, 3, 4, 5])

# คูณ 10 ให้ทุกตัวพร้อมกันทันที
print(data * 10)
```



[10 20 30 40 50]

Pandas: ผู้จัดการวัตถุดิบ (The Super-Excel)

เครื่องมือจัดการข้อมูลแบบตาราง (Tabular Data) ที่ทรงพลังที่สุด

Core Object: DataFrame

มีแถว (Rows) และคอลัมน์ (Columns) เหมือน Excel
แต่รับมือข้อมูลได้เร็วกว่าและยืดหยุ่นกว่า



```
import pandas as pd
```

```
# 1. โหลดของเข้าครัว (Load Data)
```

```
df = pd.read_csv('menu.csv')
```

```
# 2. เช็คสต็อกวัตถุดิบ (Inspect)
```

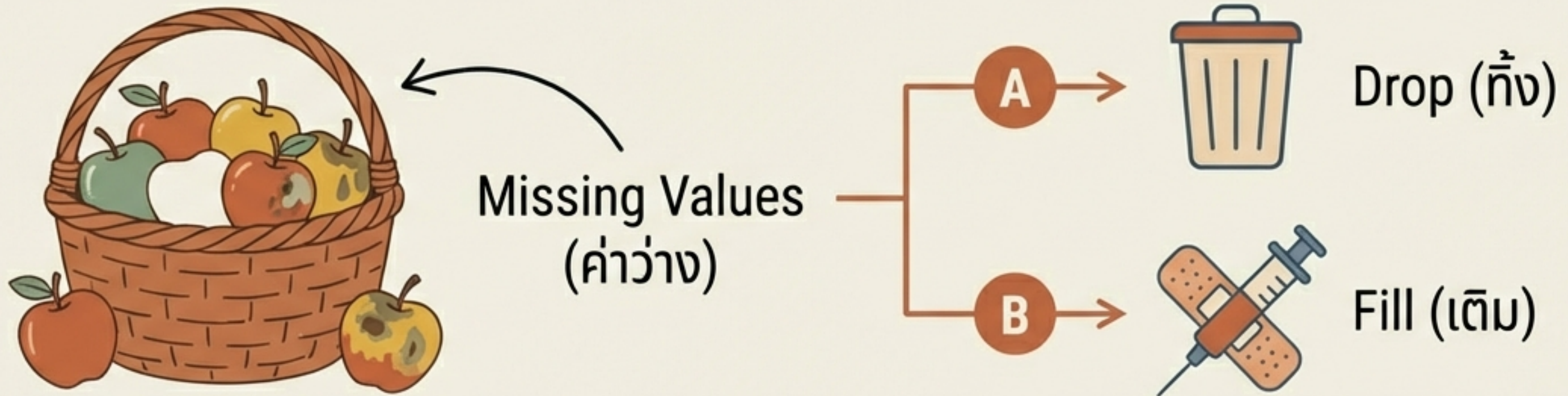
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 100 entries, 0 to 99  
Data columns (total 3 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   Item        100 non-null    object  
1   Price       100 non-null    float64  
2   Category    100 non-null    object  
dtypes: float64(1), object(2)  
memory usage: ...
```

ผลลัพธ์
(Abstract
Output)

Data Cleaning: การล้างและคัดแยกวัตถุผิดปกติ

Data Scientist ใช้เวลา 60-70% ไปกับการเตรียมข้อมูล (Cleaning)!



```
# เช็คว่ามีวัตถุผิดปกติไหนน่าเสีย (เป็นค่าว่าง)  
print(df.isnull().sum())  
  
# วิธีแก้ไข: เติมค่าว่างด้วย 0 (Fill)  
df.fillna(0, inplace=True)
```


Matplotlib: กระดานสเก็ตซ์ภาพ



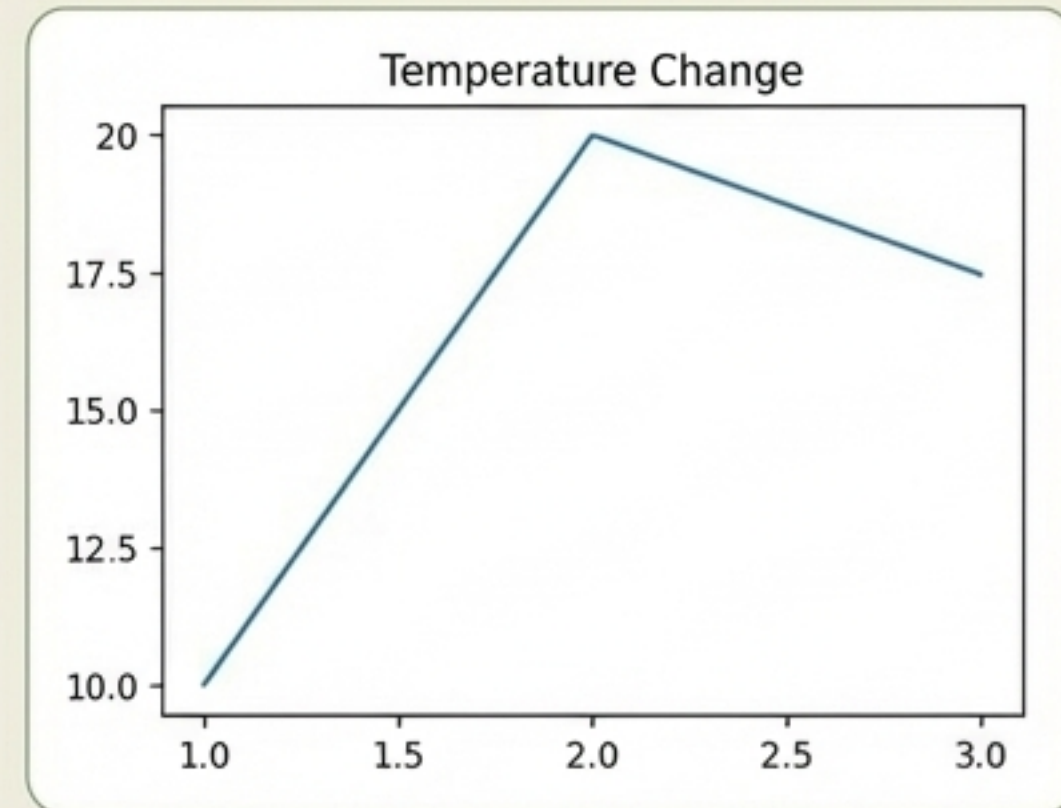
The Sketchpad: Library ที่เป็น 'พ่อทุกสถาบัน' ของการสร้างกราฟ

Pros: ยืดหยุ่นสูง ปรับแต่งได้ทุกจุด

Cons: อาจต้องเขียนโค้ดยาวเพื่อความสวยงาม

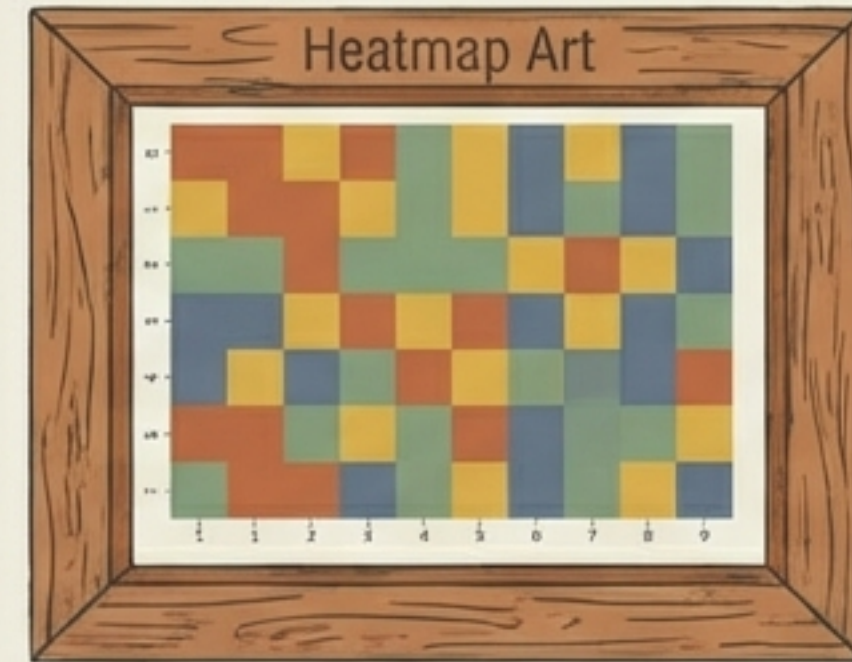
```
import matplotlib.pyplot as plt

# วาดกราฟเส้นง่ายๆ
plt.plot([1, 2, 3], [10, 20, 15])
plt.title("Temperature Change")
plt.show()
```



Seaborn: หอศิลป์แห่งข้อมูล (The Art Gallery)

สร้างบนพื้นฐานของ Matplotlib แต่เน้น
ความสวยงามทางสถิติ (Statistical Plots)
เขียนโค้ดสั้นกว่า แต่ได้กราฟที่ดูดีทันที



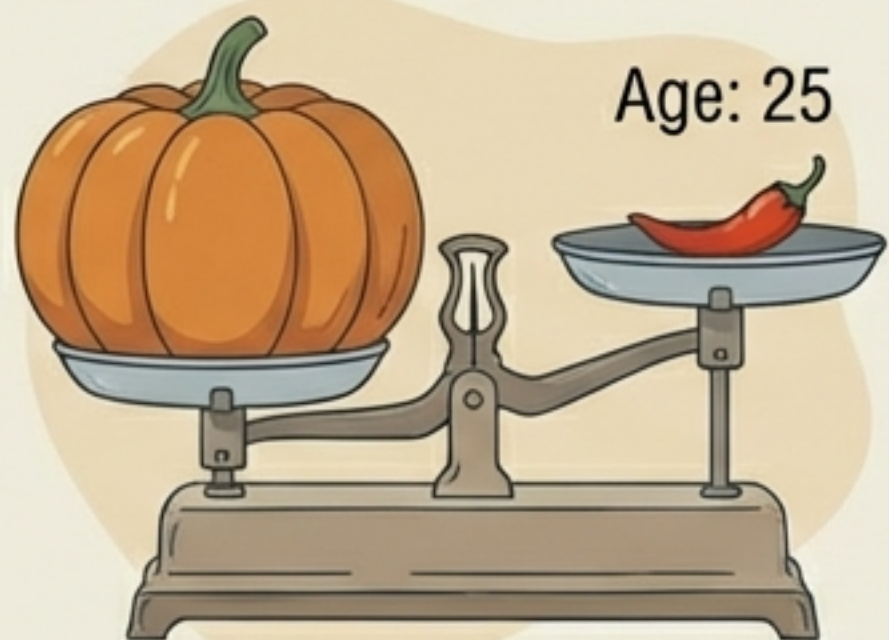
```
import seaborn as sns
```

```
# สร้าง Heatmap ดูความสัมพันธ์ (Correlation)  
sns.heatmap(df.corr(), annot=True)
```



Feature Scaling: การปรับรงรสให้กลมกล่อม

Salary: 1,000,000



Age: 25

ถ้าไม่ปรับสเกล ตัวเลขมากจะมีอิทธิพลเกินจริง
ทำให้โมเดลสับสน

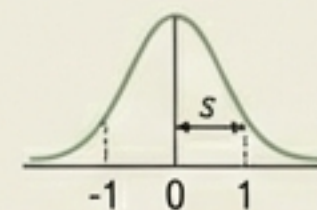
MinMax Scaler

บีบข้อมูลทั้งหมดให้อยู่ระหว่าง 0 ถึง 1



Standardization

ปรับให้ Mean=0, SD=1



```
from sklearn.preprocessing import MinMaxScaler  
scaler = MinMaxScaler()
```

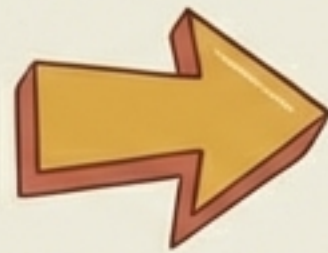
```
# ปรับสเกลข้อมูลให้สมดุล
```

```
scaled_data = scaler.fit_transform(df[['age', 'salary']])
```


Encoding: แปลภาษาให้เครื่องจักรเข้าใจ

โมเดลคำนวณได้แค่ตัวเลข ไม่เข้าใจคำว่า 'สีแดง' หรือ 'สีเขียว'

Color
Red
Green
Red



Color_Red	Color_Green
1	0
0	1
1	0

One-Hot Encoding (เหมือนเปิดสวิตช์ไฟ)

```
# แปลงข้อความให้เป็นตัวเลข  
pd.get_dummies(df['color'])
```

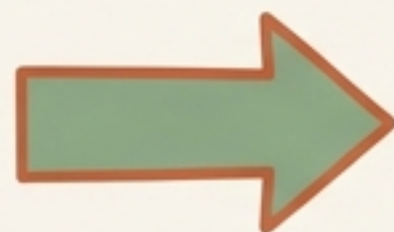

Scikit-Learn: เชฟมือโปร (The Master Chef)

ความสม่ำเสมอของ Estimator API



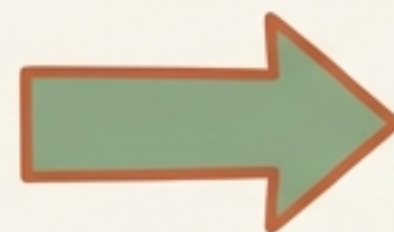
เรียกใช้โมเดล

```
`model = Model()`
```



เรียนรู้จากข้อมูล

```
`fit(X, y)`
```



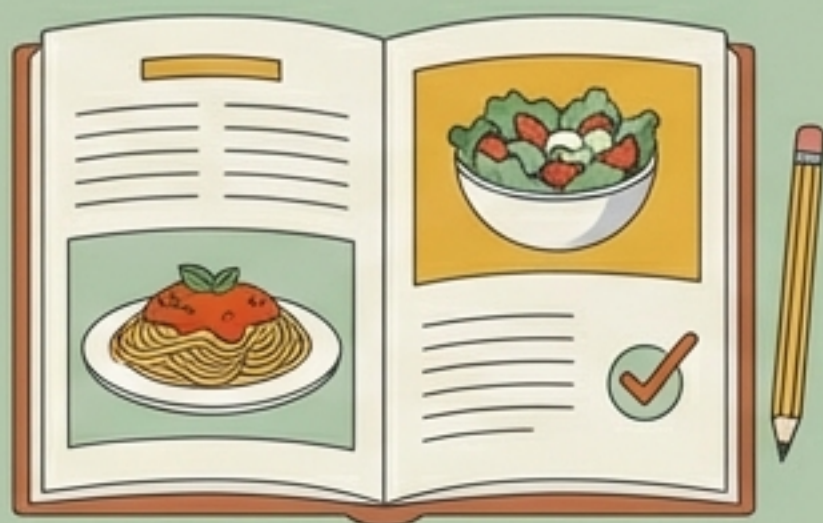
ทำนายผล

```
`predict(X_new)`
```

รูปแบบคำสั่งที่เป็นมาตรฐานเดียวกัน ช่วยให้เปลี่ยนโมเดลได้ง่ายและเรียนรู้เร็ว.

ประเภทของเมนู: Supervised vs Unsupervised

Supervised (มีเฉลย)



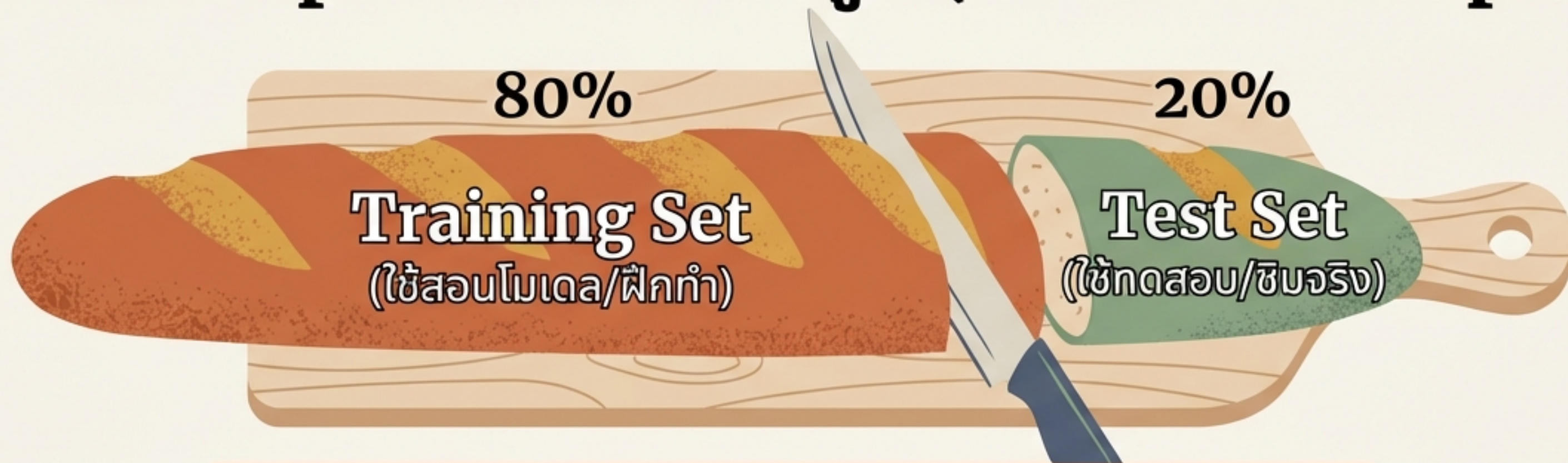
- **Classification:** ทายหมวดหมู่ (หมา/แมว)
- **Regression:** ทายตัวเลข (ราคาบ้าน)

Unsupervised (ไม่มีเฉลย)



- **Clustering:** จัดกลุ่มข้อมูล (แบ่งกลุ่มลูกค้า)
- **Goal:** ค้นหารูปแบบที่ซ่อนอยู่ (Hidden Patterns)

The Recipe: การแบ่งข้อมูล (Train-Test Split)



กฎเหล็ก: โมเดลต้อง **ไม่เคยเห็น** Test Set มาก่อน เพื่อป้องกันการจำข้อมูล (Overfitting)

```
from sklearn.model_selection import train_test_split
```

```
# แบ่งข้อมูลเป็นชุดฝึกและชุดสอบ
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```


Baking & Tasting: อบและชิมรสชาติ

Step 1: Training (อบ)



```
model.fit(X_train, y_train)
```

สอนโมเดลให้เรียนรู้ความสัมพันธ์

Step 2: Evaluation (ชิม)



```
print(model.score(X_test, y_test))
```

วัดความแม่นยำจากข้อมูลที่ไม่เคยเห็น

Full Code Example

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train) # 1. สอน (Train)
print(model.score(X_test, y_test)) # 2. ให้คะแนน (Evaluate)
```


จรรยาบรรณเชฟ: ความรับผิดชอบและก้าวต่อไป



The Responsible Chef

- ✂ ความโปร่งใส (Transparency)
- ✂ ความเป็นส่วนตัว (Privacy)
- ✂ ระวังอคติของข้อมูล (Bias)

พื้นฐานในวันนี้ (NumPy, Pandas, Scikit-Learn)
คือกุญแจสำคัญที่จะไขประตูสู่เรื่องที่ยากขึ้น
ขอให้สนุกกับการปรุงข้อมูล!